PROGRAMMING FOR PROBLEM SOLVING

UNIT 2 Problem Decomposition, Functions and Pointers

PREMANAND S

Assistant Professor

Department of Computational Intelligence

SRM Institute of Science and Technology

Kattankulathur Campus

First Year Engineering

Scenario-Based Problems

Functions in C

Think $\rightarrow$ *Analyse* $\rightarrow$ *Design* $\rightarrow$ *Code*

Real-World Function Design Problems

Problem 1 — College Welcome Screen

Scenario

A college application needs to display a standard welcome message whenever a student starts the application.

The same message should be displayed from different parts of the program.

- The message is fixed.
- It does not depend on any student information.
- Nothing needs to be sent back after displaying it.

Your Task

Design a suitable function and write a C program to demonstrate its use.

Problem 1 — Expected Output

```
===== WELCOME TO ABC COLLEGE =====

Have a great learning experience!
```

Think Before You Code

- Does the function need any information from `main()`?
- Does `main()` need any result from the function?
- What should the function receive?
- What should it return?

Problem 1 — Function Design

Question	Answer
Input required?	No
Parameters?	No
Return value?	No
Return type	`void`

Function Type

No Parameters + No Return Value

Think:

```
void welcome()
```

C Function Structure

```
#include <stdio.h>
void welcome();
int main()
{
  welcome();
  return 0;
}
void welcome()
{
  printf("===== WELCOME TO ABC COLLEGE
      =====\n");
  printf("Have a great learning experience!\n");
}
```

Problem 2 — Electricity Bill Display

Scenario

A college electricity-monitoring application needs to display the electricity bill for a particular month.

The **number of units consumed** is available in the main program.

Billing Rules

- Up to 100 units $\rightarrow$ Rs. 2 per unit
- 101–200 units $\rightarrow$ Rs. 3 per unit
- Above 200 units $\rightarrow$ Rs. 5 per unit

Example

Units Consumed: 150

Electricity Bill = Rs. 450

Problem 2 — Your Task

- Create a suitable function.
- The function should receive the number of units.
- Calculate the bill inside the function.
- Display the calculated bill.
- The main program does not need a value back.

Important

This problem combines:

Function Design + Decision Making

Think Before You Code

- What information does the function need?
- Should the calculation happen inside the function?
- Does the function need to send a result back?
- What should be passed when the function is called?

Problem 2 — Function Design

Question	Answer
Input required?	Number of units
Parameters?	Yes
Return value?	No
Return type	`void`

Function Type

Parameters + No Return Value

```
void calculateBill(int units)
```

Problem 2 – Prototype, Call and Definition

```
#include <stdio.h>
void calculateBill(int units);
int main()
{
   int units = 150;
   calculateBill(units);
   return 0;
}
```

Problem 2 – Prototype, Call and Definition

```
void calculateBill(int units)
{
  int bill;

  if (units <= 100)
  bill = units * 2;
  else if (units <= 200)
  bill = units * 3;
  else
  bill = units * 5;

  printf("Electricity Bill = Rs. %d\n", bill);
}
```

Problem 3 — Student Result Analyzer

Scenario

A college wants to calculate a student's average mark.

The main program already has the marks of **three subjects**.

Create a suitable function that:

- receives the three marks,
- calculates the average,
- sends the calculated average back to `main()`.

Important

The function should **not directly print the average**. The returned result must be used by `main()`.

Problem 3 — Expected Output

```
Subject 1 :  78
Subject 2 :  82
Subject 3 :  90

Average :  83.33
```

Think Before You Code

- What information does the function need?
- Where are the three marks available?
- Where should the average be calculated?
- Who needs the calculated average?
- Should the function display the answer or return it?

Problem 3 — Function Design

Question	Answer
Input required?	Three marks
Parameters?	Yes
Return value?	Yes
Return type	`float`

Function Type

Parameters + Return Value

```
float calculateAverage(float, float, float)
```

Problem 3 – Prototype, Call and Definition

```
#include <stdio.h>
float calculateAverage(float mark1,
float mark2,
float mark3);

int main()
{
   float mark1 = 78;
   float mark2 = 82;
   float mark3 = 90;
   float average;

   average = calculateAverage(mark1, mark2,
       mark3);

   printf("Average : %.2f\n", average);

   return 0;
}
```

Problem 3 – Prototype, Call and Definition

```
float calculateAverage(float mark1,
float mark2,
float mark3)
{
  float average;

  average = (mark1 + mark2 + mark3) / 3;

  return average;
}
```

Problem 4 — Student ID Generator

Scenario

A college admission program needs to generate a temporary student ID.

The ID should be generated **inside the function** using:

$$\boxed{\text{Student ID} = 1000 + \text{Admission Number}}$$

- The function must obtain the admission number itself.
- The admission number is not available in `main()` before the call.
- The function must return the generated ID.

Problem 4 — Expected Output

```
Enter admission number:  25

Generated Student ID = 1025
```

Think Before You Code

- Does `main()` have information to send?
- Where will the admission number be read?
- Does the function produce a result?
- Who needs that result?
- What should happen when the function finishes?

Problem 4 — Function Design

Question	Answer
Input from `main()`?	No
Parameters?	No
Input inside function?	Yes
Return value?	Yes
Return type	`int`

Function Type

No Parameters + Return Value

```
int generateStudentID()
```

Problem 4 – Prototype, Call and Definition

```
#include <stdio.h>

int generateStudentID();

int main()
{
  int studentID;

  studentID = generateStudentID();

  printf("Generated Student ID = %d\n",
      studentID);

  return 0;
}
```

Problem 4 – Prototype, Call and Definition

```
int generateStudentID()
{
   int admissionNumber;
   int studentID;

   printf("Enter admission number: ");
   scanf("%d", &admissionNumber);

   studentID = 1000 + admissionNumber;

   return studentID;
}
```

Four Problems — Four Function Designs

Problem	Parameters	Return	Function Type
Welcome Screen	No	No	Type 1
Electricity Bill	Yes	No	Type 2
Result Analyzer	Yes	Yes	Type 3
Student ID	No	Yes	Type 4

The Key Idea

Before writing a function, ask two questions:

What should the function receive?

What should the function give back?

Think Before You Code

Every function-design problem can begin with

1. **What information does the function need?**
2. **Does it need to send a result back?**

Parameters + Return Value

Once you identify these two,
the function type becomes much easier to choose.

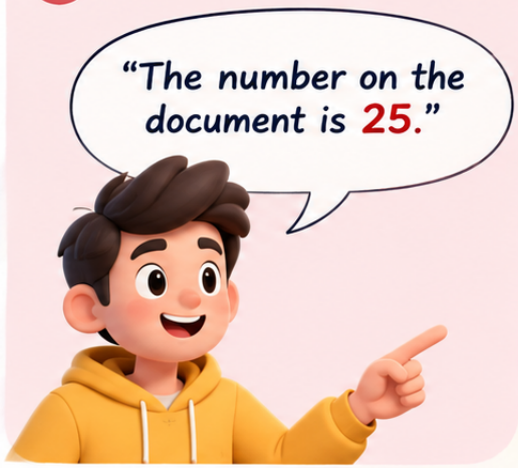
Let's Start with a Story...

Think...
What do you have?
And what do you need?

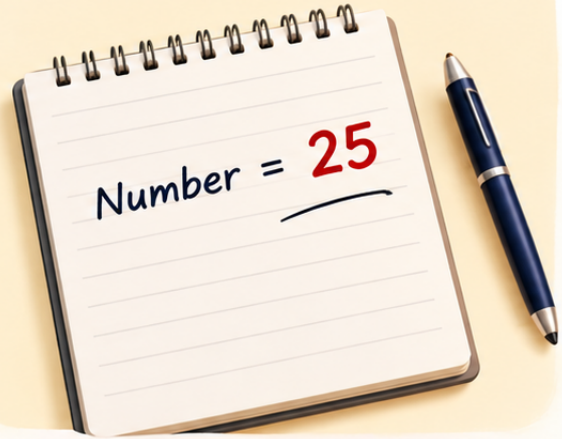
1 **Your friend has a locker.**
Inside the locker, there is an important document.



2 **Your friend tells you:**



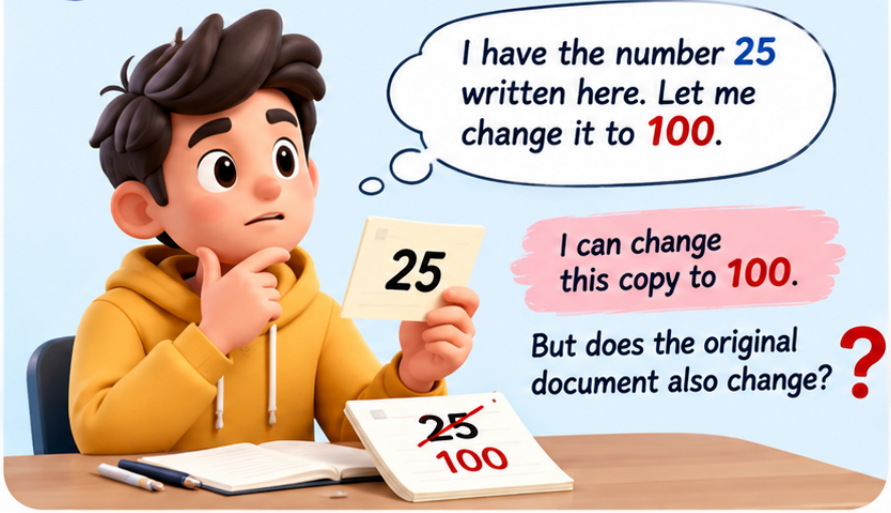
3 **You write down the number.**



Now Imagine This...

Can you change the original?

1 You only have a **copy** of the number.



2 What about the **original document?**



The document inside the locker still has 25.

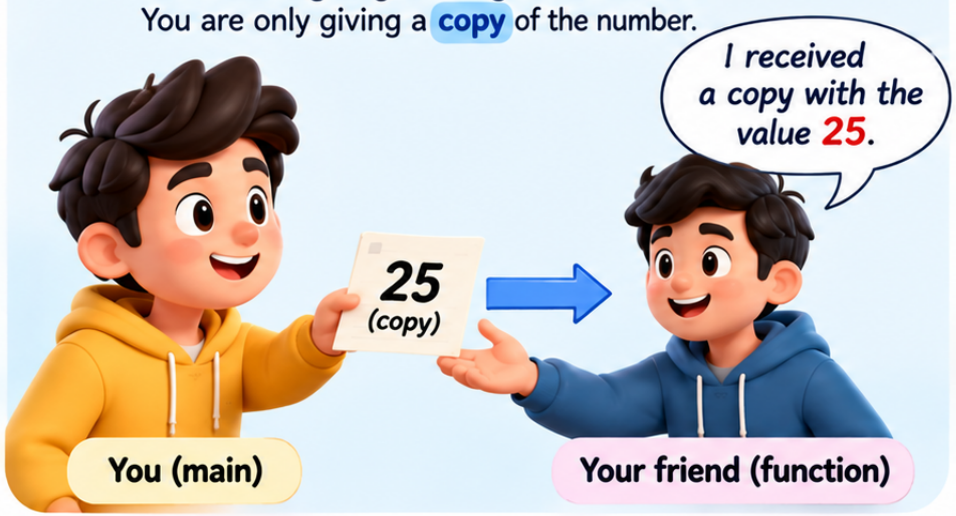
If you only have a copy of the number, can you change the number on the original document?

What Actually Happens?

The function receives a copy!

1 **You give the number to your friend.**

You are not giving the original document.
You are only giving a **copy** of the number.



2 **You still have the original document.**

The original document is untouched.
It still has **25**.



So, when we pass a variable to a function, the function receives a copy of the value.

Can You Change the Number?

You have the value... But not the original!

1 You try to change the copy.

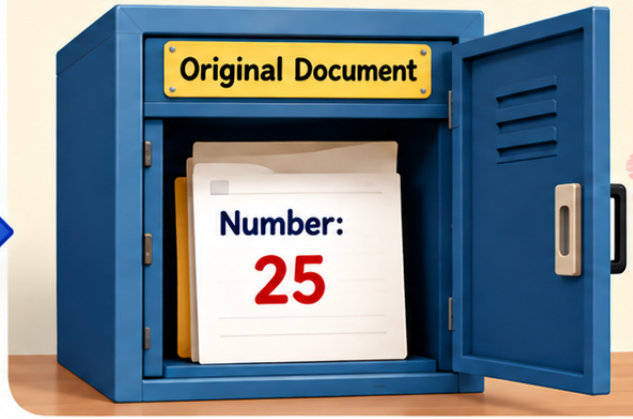
You received a copy of the number **25**.
Now you change it to **100.**



But...

2 The original document is unchanged.

The original document still has **25**, because you only changed the **copy**.



The original number is still **25**.

If you only have a copy of the value, you cannot change the original.

To change the original, we need to know **where it is stored!**

Pass by Value

The function receives a copy of the value

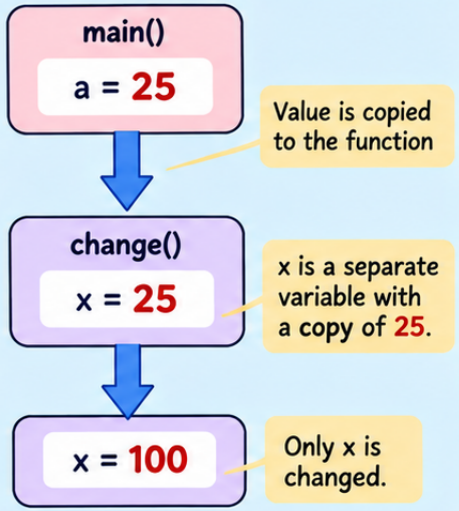
You gave the number, not the original document!

Real-Life Analogy



Your friend only has a **copy** of the number. He can change his copy, but your original number remains the same.

What Actually Happens?



C Program Example

```c
#include <stdio.h>

void change(int x)
{
    x = 100;  // Changes only the copy
}
int main()
{
    int a = 25;
    change(a);  // Passing a copy of a
    printf("a = %d\n", a);
    return 0;
}
```

Output

```
a = 25
```

The original value remains unchanged!

Key Takeaway

When we pass a variable to a function, the function receives a **copy of the value**. Any changes made inside the function do not affect the original variable.

A Visual View Inside the Function

The value is copied from main() to the function.

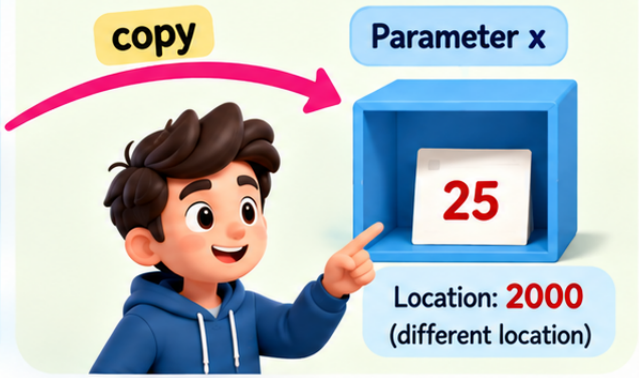
1 In main()

Variable **a** has a value **25**.



2 When we call the function

A **copy** of **25** is passed to the function.



3 Inside the function

We change the value of **x** to **100**.



So, what will be the value of a after the function call?



Pass by Value in C

A copy of the value is passed to the function.

1 Example Program

```c
#include <stdio.h>

void change(int x)
{
    x = 100;
}

int main()
{
    int a = 25;

    change(a);

    printf("a = %d\n", a);
    return 0;
}
```

2 What Happens?

Step 1 In main()

Variable **a** has value **25** at location **1000**.



Step 2 Function call change(a)

The value **25** is **copied** and passed to the function.

Copy of 25



Step 3 Inside the function

We change **x** to **100**. This changes **only the copy**.



3 After the function call

The original variable **a** in main() is **unchanged**.



4 Output

```
a = 25
```

In pass by value, the function receives a copy of the value.
Changing the parameter does not change the original variable.

Pass by Value in C

A copy of the value is passed to the function.

1 Example Program

```c
#include <stdio.h>

void change(int x)
{
    x = 100;
}

int main()
{
    int a = 25;

    change(a);

    printf("a = %d\n", a);
    return 0;
}
```

2 What Happens?

Step 1 In main()

Variable **a** has value **25** at location **1000**.



Step 2 Function call change(a)

The value **25** is **copied** and passed to the function.

Copy of 25



Step 3 Inside the function

We change **x** to **100**. This changes **only the copy**.



3 After the function call

The original variable **a** in main() is **unchanged**.



4 Output

```
a = 25
```

In pass by value, the function receives a copy of the value.
Changing the parameter does not change the original variable.

From Functions to the Next Question

Let's Think Before We Learn Something New...

Bridge Question 1

When we call a function and pass a variable to it,

Does the function receive the original variable or only a copy of its value?

Bridge Question 2

If the function changes the value it receives,

Will the original variable in `main()` also change?

Predict the Output

```
#include <stdio.h>
void change(int x)
{
 x = 100;
}
int main()
{
 int a = 10;
 change(a);
 printf("a = %d", a);
 return 0;
}
```

What Actually Happened?

We passed `a` to the function...

Question

If `a = 10`, what exactly reaches the function?

The variable `a`?

OR

The value 10?

Think

If the function receives only a copy of the value,

How can it ever change the original variable?

Pass by Value

The Function Receives a Copy

`a = 10`

↓

change(a)

↓

`x = 10`

Key Idea

The value of `a` is copied into `x`.

Changing `x` changes only the copy.

The original `a` remains unchanged.

Practice Question 1 — Change the Value

Problem Statement

Write a C program with a function `changeValue()` that receives an integer.

The function should:

- receive an integer value
- change the received value to **100**

In `main()`:

- initialize a variable with **25**
- call `changeValue()` by passing the variable
- display the variable after the function call

Think Before You Code

Will the original variable become **100**?

Solution — Practice Question 1

```
#include <stdio.h>
void changeValue(int x)
{
 x = 100;
}

int main()
{
 int value = 25;
 changeValue(value);
 printf("Value = %d", value);

 return 0;
}
```

Discount Calculator

Problem Statement

A shopping application needs to calculate the final price of a product after applying a discount.

Write a C program with a function `calculateDiscount()` that:

- receives the original price
- receives the discount percentage
- calculates and returns the discount amount

Example

Input:

Original Price = 2000

Discount = 15%

Output:

Discount Amount = 300

Original Price = 2000

Solution — Practice Question 3

```
#include <stdio.h>

float calculateDiscount(float price, float discount)
{
 return (price * discount) / 100;
}

int main()
{
 float price, discount, discountAmount;

 printf("Enter original price: ");
 scanf("%f", &price);

 printf("Enter discount percentage: ");
 scanf("%f", &discount);
```

Solution

```
#include <stdio.h>
float calculateDiscount(float price, float discount);
int main()
{
 float price, discount, discountAmount;
 printf("Enter original price: ");
 scanf("%f", &price);
 printf("Enter discount percentage: ");
 scanf("%f", &discount);

 discountAmount = calculateDiscount(price, discount);

 printf("Discount Amount = %.2f\n", discountAmount);
 printf("Original Price = %.2f", price);
 return 0;
}
```

Solution

```
float calculateDiscount(float price, float discount)
{
 return (price * discount) / 100;
}
```

Practice Question — Electricity Bill Calculator

Problem Statement

A power company wants to calculate the electricity bill for a customer.

Write a C program with a function `calculateBill()` that receives the **number of units consumed**.

Use the following tariff:

- First 100 units → Rs. 2 per unit
- Next 100 units → Rs. 3 per unit
- Above 200 units → Rs. 5 per unit

The function should **calculate and return** the total bill amount.

Sample Input and Output

```
Enter units consumed:  250
Bill Amount = 750
Units Consumed = 250
```

Solution — Electricity Bill Calculator

```
#include <stdio.h>

int calculateBill(int units);

int main()
{
 int units, bill;
 printf("Enter units consumed: ");
 scanf("%d", &units);

 bill = calculateBill(units);
 printf("Bill Amount = %d\n", bill);
 printf("Units Consumed = %d", units);

 return 0;
}
```

Solution — Electricity Bill Calculator

```
int calculateBill(int units)
{
 int bill;
 if (units <= 100)
 {
  bill = units * 2;
 }
 else if (units <= 200)
 {
  bill = (100 * 2) + ((units - 100) * 3);
 }
 else
 {
  bill = (100 * 2) + (100 * 3) + ((units - 200) * 5);
 }
 return bill;  }
```

Practice Question — Student Performance Analyzer

Problem Statement

A teacher wants to analyse a student's marks in **three subjects**.

Write a C program with a function `calculateAverage()` that receives the marks of the three subjects and **returns the average**.

Based on the returned average, display:

- Average $\geq 90 \rightarrow$ `Excellent`
- Average $\geq 75 \rightarrow$ `Very Good`
- Average $\geq 50 \rightarrow$ `Good`
- Average $< 50 \rightarrow$ `Needs Improvement`

The Program Should Display

1. The average mark
2. The corresponding performance category
3. The three original marks again

Solution — Student Performance Analyzer

```
#include <stdio.h>
float calculateAverage(int m1, int m2, int m3);
int main()
{
 int m1, m2, m3;
 float average;

 printf("Enter marks: ");
 scanf("%d %d %d", &m1, &m2, &m3);

 average = calculateAverage(m1, m2, m3);
 printf("Average = %.2f\n", average);
 if (average >= 90)
  printf("Performance = Excellent");
 else if (average >= 75)
  printf("Performance = Very Good");
```

Solution — Student Performance Analyzer

```
 else if (average >= 50)
  printf("Performance = Good");
 else
  printf("Performance = Needs Improvement");

 printf("\nMarks = %d %d %d", m1, m2, m3);

 return 0;
}

float calculateAverage(int m1, int m2, int m3)
{
 return (m1 + m2 + m3) / 3.0;
}
```

Practice Question — ATM Withdrawal Validator

Problem Statement

An ATM receives a customer's **account balance** and **withdrawal amount**.

Write a C program with a function `canWithdraw()` that receives the balance and withdrawal amount.

The withdrawal is allowed only when:

- Withdrawal amount is greater than **0**.
- Withdrawal amount does not exceed the **available balance**.

If the withdrawal is allowed, calculate and display the **remaining balance**. The original balance should also be displayed after the function call.

Solution — ATM Withdrawal Validator

```
#include <stdio.h>

int canWithdraw(int balance, int withdrawal);

int main()
{
 int balance, withdrawal;
 int result;

 printf("Enter balance: ");
 scanf("%d", &balance);

 printf("Enter withdrawal amount: ");
 scanf("%d", &withdrawal);

 result = canWithdraw(balance, withdrawal);
```

Solution — ATM Withdrawal Validator

```
 if (result == 1)
 {
  printf("Withdrawal Allowed\n");

  printf("Remaining Balance = %d\n",
  balance - withdrawal);
 }
 else
 {
  printf("Withdrawal Rejected\n");
 }

 printf("Original Balance = %d", balance);

 return 0;
}
```

Solution — ATM Withdrawal Validator

```
int canWithdraw(int balance, int withdrawal)
{
 if (withdrawal > 0 && withdrawal <= balance)
 return 1;
 else
 return 0;
}
```

Practice Question — Temperature Converter

Problem Statement

Write a C program with a function `convertTemperature()` that receives a temperature in Celsius and **returns the equivalent temperature in Fahrenheit**.

Use the formula:

$$F = (C \times 9/5) + 32$$

In `main()`:

1. Read the temperature in Celsius.
2. Call `convertTemperature()` by passing the temperature.
3. Display the temperature in Fahrenheit.
4. Display the original Celsius temperature again.

Practice Question — Number Analyzer

Problem Statement

Write a C program with a function `analyzeNumber()` that receives an integer and **returns**:

- `1` if the number is positive
- `0` if the number is zero
- `-1` if the number is negative

In `main()`:

1. Read an integer.
2. Call `analyzeNumber()` by passing the number.
3. Use the returned value to display the appropriate message.
4. Display the original number again.

We Have a Problem...

So Far...

When we pass a variable to a function:

`main()` $\longrightarrow$ `function()`

the function receives a **copy of the value**.

But What If...

We want the function to **change the original variable**?

Can the function reach the original variable?

From Functions to Pointers – A Real-Life Story

Functions let us organize work. Pointers let us work with memory directly!

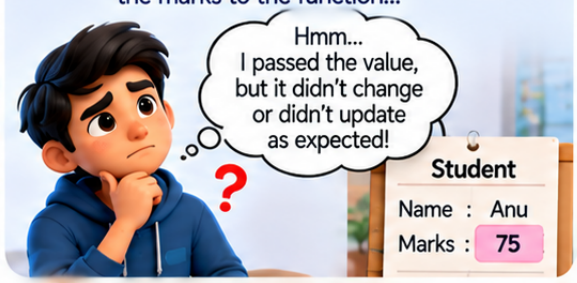
1 Arjun Learns Functions

Arjun was happy! Functions helped him organize his program. He could break a big problem into smaller tasks and reuse them.



2 A New Requirement

One day, Arjun wrote a function to update a student's marks. He passed the marks to the function...



3 The Problem

Arjun wanted the function to actually modify the original data (like changing marks, updating multiple values, or working with large data), but he realized something...



4 Real-Life Analogy

It's like giving someone a photocopy of a document and asking them to edit it.



5 But Sometimes We Need to Change the Original!

In many real-world situations, we want a function to **directly work with the actual data**:

- Update the original data (e.g., modify marks, values, etc.)
- Work with large data (arrays, structures) efficiently.
- Avoid making unnecessary copies (saves memory and time).
- Access and modify data at its actual memory location.

6 This is Where Pointers Come In!

Pointers help us pass the **address** of a variable to a function, so the function can directly **access and modify the original data in memory.**



Key Takeaway:

Functions help us organize code, but **pointers** give us the power to directly work with the **actual memory location** of variables and **solve more advanced problems** in C.

Why Do We Need a Pointer?

Let's think about a real variable...

A Variable Has More Than Just a Value

Suppose:

```
int marks = 25;
```

The variable has:

A Value + **A Memory Address**

The Idea

If we give the function the **address** of the original variable,

the function can reach the original variable.

How Do We Find the Address?

We now know that the function may need the **location** of a variable.

In C

Suppose we have:

```
int marks = 25;
```

Question

How can we ask C:

"Where is `marks` stored in memory?"

The Address-of Operator &

Suppose we have a variable

```
int marks = 25;
```

`&marks`

What does `&marks` mean?

The memory address of the variable `marks`

`marks` $\rightarrow$ **Value**

`&marks` $\rightarrow$ **Address**

We Have the Address... Now What?

We now know how to find the address:

`&marks`

But Here Is the Next Question

Where do we store this address?

Think!

Just like we use a variable to store a value...

Can we have a special variable to store an address?

Meet the Pointer

A Special Variable

A **pointer** is a variable that stores the **address of another variable**.

Suppose:

```
int marks = 25;
```

We can create a pointer:

```
int *p;
```

Think

`marks` stores a **value**

`p` stores an **address**

Connecting the Pointer to the Variable

We Have

```
int marks = 25;

int *p;
```

Now We Connect Them

```
p = &marks;
```

`&marks` → **address of marks**

`p` → **stores that address**

What Does *p Mean?

We Already Know

```
p = &marks;
```

p stores the **address** of `marks`.

Now Ask

If p contains an address,

how do we access the value stored at that address?

*p

*p → **value stored at the address in** p

Can We Change the Original Variable?

Remember

```
int marks = 25;
int *p = &marks;
```

We know:

`p` $\rightarrow$ address of `marks`

`*p` $\rightarrow$ value stored at that address

Now Try This

```
*p = 100;
```

What will happen to `marks`?

Can We Change the Original Variable?

Remember

```
int marks = 25;
int *p = &marks;
```

We know:

`p` $\rightarrow$ address of `marks`

`*p` $\rightarrow$ value stored at that address

Now Try This

```
*p = 100;
```

What will happen to `marks`?

What Just Happened?

Before

```
marks = 25
```

The variable contains the value **25**.

After

```
*p = 100;
```

The value stored in the original variable becomes **100**.

The Important Idea

*p gives us access to the **original variable**.

No copy was changed.

The original variable was changed.

Can a Function Receive an Address?

So far, we have seen:

Pass by Value

Value $\longrightarrow$ Function

Function works with a **copy**

Now We Know Something New

Address $\longrightarrow$ Function

The function can use that address to access **the original variable**.

But how do we pass an address to a function?

Passing the Address to a Function

Suppose we have

```
int marks = 25;
```

Instead of passing the value:

```
change(marks);
```

we can pass the **address**:

```
change(&marks);
```

What Reaches the Function?

Not the value alone...

The address of the original variable.

Now Put Everything Together

Step 1 — In `main()`

```
int marks = 25;

change(&marks);
```

Step 2 — Function Receives the Address

```
void change(int *p)
```

Step 3 — Access the Original

```
*p = 100;
```

Result

`marks` changes from **25** to **100**.

Pass by Value vs Pass by Reference

Pass by Value

Value is passed

```
change(x);
```

Function receives a **copy**.

Change the copy

↓

Original unchanged

Pass by Reference

Address is passed

```
change(&x);
```

Function receives the **address**.

Access original

↓

Original can change

The Key Difference

Value → Copy **Address** → Original can be accessed

Practice Question — Update the Value

Problem Statement

Write a C program with a function `updateValue()` that receives the **address of an integer**.

Inside the function:

- Change the value of the variable to **100**.

In `main()`

1. Initialize an integer variable with **25**.
2. Display its value before the function call.
3. Call `updateValue()` by passing the **address of the variable**.
4. Display its value after the function call.

Solution — Update the Value

```
#include <stdio.h>
void updateValue(int *p);
int main()
{
 int value = 25;
 printf("Before = %d\n", value);
 updateValue(&value);
 printf("After = %d", value);
 return 0;
}
void updateValue(int *p)
{
 *p = 100;
}
```

Medium Practice Question — Swap Two Numbers

Problem Statement

Write a C program with a function `swap()` that receives the **addresses of two integers**.

The function should **swap the values of the two original variables**.

In `main()`

1. Read two integers.
2. Display their values before the function call.
3. Call `swap()` by passing the **addresses of the two variables**.
4. Display their values after the function call.

Solution — Swap Two Numbers

```
#include <stdio.h>
void swap(int *x, int *y);
int main()
{
 int a, b;
 printf("Enter two numbers: ");
 scanf("%d %d", &a, &b);
 printf("Before Swap:\n");
 printf("a = %d\n", a);
 printf("b = %d\n", b);
 swap(&a, &b);
 printf("\nAfter Swap:\n");
 printf("a = %d\n", a);
 printf("b = %d", b);
 return 0;
}
```

Solution — Swap Two Numbers

```
void swap(int *x, int *y)
{
 int temp;

 temp = *x;
 *x = *y;
 *y = temp;
}
```

HOTS Practice Question — Bill Adjustment System

Problem Statement

A shopping bill contains the total amount paid by a customer.
Write a C program with a function `applyDiscount()` that receives the **address of the bill amount**.

The function should:

- If the bill is ≥ 5000, apply a **20% discount**.
- If the bill is ≥ 2500 but less than 5000, apply a **10% discount**.
- Otherwise, apply **no discount**.

The function must update the **original bill amount**.

In `main()`

1. Read the bill amount.
2. Display the bill before applying the discount.
3. Call `applyDiscount()` by passing its address.
4. Display the updated bill amount.

Solution – Bill Adjustment System

```
#include <stdio.h>

void applyDiscount(float *bill);
int main()
{
 float bill;
 printf("Enter bill amount: ");
 scanf("%f", &bill);
 printf("Before Discount = %.2f\n", bill);

 applyDiscount(&bill);

 printf("After Discount = %.2f", bill);
 return 0;
}
```

Solution – Bill Adjustment System

```
void applyDiscount(float *bill)
{
 if (*bill >= 5000)
 *bill = *bill - (*bill * 20 / 100);
 else if (*bill >= 2500)
 *bill = *bill - (*bill * 10 / 100);
}
```